GeneWeaver

Project Overview • 3 Members • 20 Days

1. About the Project

CRISPR is a gene-editing tool. Sometimes it cuts DNA at the wrong place, at a spot that looks similar to the target. These are called **off-target** sites. GeneWeaver takes a short guide sequence (about 20 letters of A, C, G, T), scans a huge DNA file, finds places that *almost* match, and ranks them by risk. A GPU is used to make the scan much faster than normal Python.

How the project flows:

Step 1	Step 2	Step 3	Step 4
Read the FASTA file, clean it, split into chunks **(Member 1)**	Find mismatches on CPU and GPU **(Member 2)**	Run chunks in parallel, score and rank the hits **(Member 3)**	Show the ranked results table **(Member 1)**

Key words

Word	Simple meaning
FASTA	A text file that stores DNA letters (A, C, G, T)
Chunk	A small piece of a very large DNA sequence
Mismatch	A letter where the guide and the DNA do not match
GPU	A processor that does thousands of small jobs at the same time
Dask	A tool that runs many chunks in parallel
Score	A risk number: fewer mismatches means higher risk

2. Work of Each Member

Member 1 - Data Pipeline + Results Display

Overview: Prepares the DNA data for the other members and shows the final results on screen.

Things to do:

- Read and validate .fasta files (BioPython)
- Clean the data: handle N bases and lowercase letters
- Convert letters to numbers: A=0, C=1, G=2, T=3
- Split the DNA into overlapping chunks
- Show a progress bar and a results table (position, mismatches, score, highlighted bases)
- Create main.py to run the whole project

Learn first: Python file handling, NumPy, FASTA format, BioPython, Rich library

Member 2 - GPU / CUDA Developer

Overview: Builds the part that finds mismatches, first on CPU and then on GPU, and shows how much faster the GPU is.

Things to do:

- Build a CPU version: slide the guide along the DNA and count mismatches
- Measure the CPU time
- Write the Numba CUDA kernel (@cuda.jit)
- Handle data transfer between CPU and GPU
- Optimize the GPU code
- Create the CPU vs GPU benchmark table and chart

Learn first: CPU vs GPU, parallel processing, Numba, CUDA basics, GPU memory, benchmarking

Member 3 - Dask + Scoring + Integration

Overview: Decides how risky each match is, runs the work in parallel, and joins all the parts together.

Things to do:

- Write the scoring function (mismatch count plus extra weight near the PAM site)
- Use Dask to process chunks in parallel
- Combine Dask with the GPU code and rank the hits
- Write tests for the parser, aligner and scoring
- Write the README
- Prepare the final demo and slides

Learn first: Dask, NumPy, scoring logic, pytest

3. Final Output

When the project is finished, we give a FASTA file and a guide sequence to the program, and it shows a ranked list of possible off-target sites on screen, with a chart that compares CPU speed and GPU speed.

> If you do not understand anything about the project, just text me and I will help you understand it.